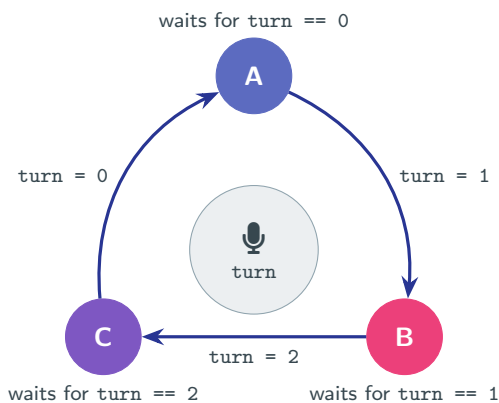


Computer Hardware ■ Lecture 8: Practice with Threads

Task 7: Threads take turns

Three threads must print in turn: A B C A B C ... Now each of them asks again and again whether it is its turn. Make them sleep instead.

This task uses condition variables: read the introduction in Task 6 first. In Task 6 every player waited for the **same** thing (all players are here). Here every thread waits for something **different**: thread A waits for `turn == 0`, thread B for `turn == 1`, thread C for `turn == 2`.



the output, one letter every 300 ms:

A B C A B C A ...

While one thread speaks, the other two ask again and again: "is it my turn?"

Ready code (in `turns.h`, we do not look inside)

Functions

`say(name)` print the letter: 300 ms

Constants

`ROUNDS` 10: every thread says its letter 10 times

The code

The template is in [Task7](#). This is `turns.cpp`:

```
const int THREADS = 3;

int turn = 0;           // whose turn it is: 0 = A, 1 = B, 2 = C
std::mutex m;

void player(int id) {
    char name = 'A' + id;
    for (int i = 0; i < ROUNDS; i++) {
        m.lock();
        while (turn != id) {    // not my turn? ask again
            m.unlock();
            m.lock();
        }
        m.unlock();
        say(name);              // takes 300 ms: not under the mutex
        m.lock();
        turn = (turn + 1) % THREADS;
        m.unlock();
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < THREADS; id++)
        threads.emplace_back(player, id);
    for (auto &t : threads)
        t.join();
    printf("\n");
}
```

In the task folder, make `run` compile the program and runs it.

Your tasks

- 1 Run it with `time ./build/turns`. How much processor time (user) did the threads use, when they mostly waited for their turn?
- 2 Replace busy waiting with a `std::condition_variable`. Run it with `time` again.
- 3 Try `notify_one()`, then `notify_all()`. Run each version several times. Which one always works? Why?
- 4 With 2 threads (set `THREADS` to 2), `notify_one()` works every time. With 3 the program sometimes hangs. Why?

Hint for question 4

`notify_one()` wakes up **one** of the waiting threads, but you do not choose which one.